TÉCNICAS DE HACKING

Práctica 2: Reconocimiento Activo

Autor: Rodrigo Revuelta Alonso

Profesor: Alfredo Robledano Abasolo

Universidad Europea de Madrid

Fecha: 26 de abril de 2026

Resumen

Este trabajo describe el proceso de descubrimiento de hosts y análisis de puertos en una red simulada, utilizando herramientas de uso habitual en el ámbito de la ciberseguridad ofensiva. La práctica se divide en dos partes diferenciadas.

En la primera parte, se ha desarrollado una función en Python llamada `craft_discovery_pkts` que permite construir y enviar paquetes de red con tres protocolos distintos: UDP, TCP ACK e ICMP Timestamp. Esta función ha sido utilizada para detectar qué equipos están activos en una red simulada con contenedores Docker, mostrando los resultados de forma clara a través de un Jupyter Notebook.

En la segunda parte, se ha analizado en profundidad el comportamiento por defecto de la herramienta Nmap cuando se ejecuta para descubrir puertos abiertos. Se han estudiado los paquetes enviados y recibidos, el número de puertos escaneados y los mecanismos mediante los cuales Nmap determina el estado de cada puerto.

Todas las pruebas han sido realizadas en un entorno seguro y controlado, desplegado con Docker sobre un sistema Kali Linux, sin afectar en ningún momento a sistemas o redes reales.

Palabras clave: reconocimiento activo, Scapy, Nmap, host discovery, port scanning, Docker, Wireshark.

Índice

Índice

1. Introducción 5
   1.1. Contexto y motivación 5
   1.2. Objetivos de la práctica 5
   1.3. Estructura del documento 5
   1.4. Marco ético y legal 5
2. Entorno de laboratorio 6
   2.1. Infraestructura utilizada 6
   2.2. Configuración de la red Docker 6
   2.3. Hosts del laboratorio 6
   2.4. Gestión de dependencias 6
3. Descubrimiento de hosts con Scapy 8
   3.1. Introducción a Scapy 8
   3.2. Protocolos utilizados 8
      3.2.1. UDP (User Datagram Protocol) 8
      3.2.2. TCP ACK 8
      3.2.3. ICMP Timestamp Request 8
   3.3. Implementación de `craft_discovery_pkts` 9
      3.3.1. Diseño de la función 9
      3.3.2. Lógica de construcción de paquetes 9
      3.3.3. Función auxiliar `discover_hosts` 9
   3.4. Demostración en Jupyter Notebook 10
   3.5. Análisis del tráfico generado 10
      3.5.1. Tráfico TCP ACK 11
      3.5.2. Tráfico ICMP Timestamp 12
      3.5.3. Tráfico UDP sin respuesta 13
4. Comportamiento por defecto de Nmap y estado de puertos 14
   4.1. Introducción a Nmap 14
   4.2. Estado de un puerto 14
      4.2.1. Puerto abierto 14
      4.2.2. Puerto cerrado 14
      4.2.3. Puerto filtrado 14
   4.3. TCP SYN Scan: el escaneo por defecto de Nmap 15
   4.4. Puertos escaneados por defecto 15
   4.5. Opciones utilizadas en la práctica 15
   4.6. Resultados del escaneo 16
      4.6.1. Escaneo a 172.28.0.11 (nginx — puerto 80 abierto) 16
      4.6.2. Escaneo a 172.28.0.10 (alpine — sin puertos abiertos) 16

4.7. Análisis del tráfico de Nmap con Wireshark 16
4.7.1. Detalle de un paquete SYN 17
4.7.2. Detalle de un paquete RST/ACK (puerto cerrado) 18
4.7.3. Puerto 80 abierto — filtro Wireshark 19
5. Resultados 20
5.1. Resultados del descubrimiento de hosts con Scapy 20
5.2. Resultados del escaneo de puertos con Nmap 20
6. Conclusiones 21
7. Bibliografía 22

1. Introducción

1.1. Contexto y motivación

En el ámbito de la ciberseguridad, cualquier proceso de auditoría comienza con una fase de reconocimiento en la que el auditor recopila información sobre los sistemas objetivo. Esta fase puede dividirse en dos grandes categorías: el reconocimiento pasivo, en el que se obtiene información sin interactuar directamente con los sistemas; y el reconocimiento activo, en el que se envían paquetes de red para obtener respuestas que permitan identificar hosts, servicios y vulnerabilidades [1].

El reconocimiento activo es una etapa crítica porque, aunque proporciona información más detallada y precisa, también deja rastros en los sistemas y redes auditadas. Por este motivo, es fundamental que los auditores de seguridad comprendan en profundidad qué información generan las herramientas que utilizan, cómo interpretar los resultados y cómo minimizar el impacto de sus acciones sobre los sistemas auditados.

1.2. Objetivos de la práctica

Esta práctica persigue dos objetivos fundamentales:

- **Implementar técnicas de descubrimiento de hosts** mediante el uso de la librería Scapy [2] en Python [3], desarrollando una función modular y reutilizable que soporte múltiples protocolos de red (UDP, TCP ACK e ICMP Timestamp).
- **Analizar el comportamiento por defecto de Nmap** [1] en el escaneo de puertos, estudiando los paquetes enviados y recibidos, el número de puertos analizados y los mecanismos de determinación del estado de cada puerto.

1.3. Estructura del documento

El documento se organiza de la siguiente manera: la sección 2 describe el entorno de laboratorio utilizado; la sección 3 desarrolla la implementación del descubrimiento de hosts con Scapy; la sección 4 analiza el comportamiento de Nmap; la sección 5 presenta los resultados obtenidos; y la sección 6 recoge las conclusiones del trabajo.

1.4. Marco ético y legal

Tal y como se indica en el enunciado de la práctica, todas las técnicas de reconocimiento activo han sido aplicadas exclusivamente sobre entornos simulados mediante herramientas de contenedorización Docker [4]. No se han realizado pruebas sobre sistemas o redes reales, y todas las actividades se han llevado a cabo en un contexto académico y con fines educativos.

2. Entorno de laboratorio

2.1. Infraestructura utilizada

Para la realización de esta práctica se ha desplegado un entorno de red simulado utilizando Docker Compose [4]. La elección de Docker como plataforma de virtualización responde a varios factores: su ligereza en comparación con máquinas virtuales completas, la facilidad para definir redes y servicios mediante ficheros de configuración declarativos, y la posibilidad de reproducir el entorno de forma sencilla en cualquier sistema.

El entorno se ha desplegado sobre un sistema Kali Linux, distribución de referencia en el ámbito de la ciberseguridad ofensiva, que incluye de forma nativa herramientas como Nmap [1], Wireshark [5] y tcpdump.

2.2. Configuración de la red Docker

La red creada para la práctica recibe el nombre `labnet` y utiliza el driver `bridge`, que crea una red interna aislada entre los contenedores. Se ha configurado la subred `172.28.0.0/24`, lo que proporciona hasta 254 direcciones IP utilizables.

La interfaz de red bridge creada por Docker en el sistema host recibe el identificador `br-f88c70a20649`, y es a través de esta interfaz por donde se ha capturado el tráfico generado durante las pruebas con tcpdump y Wireshark [5].

2.3. Hosts del laboratorio

El entorno incluye los siguientes hosts:

Nombre	IP	Imagen Docker	Servicios
target1	172.28.0.10	alpine	Ninguno
target2	172.28.0.11	nginx	HTTP puerto 80
Inexistente	172.28.0.99	—	Host inactivo

Tabla 1: Hosts del entorno de laboratorio

- **target1 (alpine)**: Contenedor basado en Alpine Linux, una distribución minimalista sin servicios expuestos. Representa un host activo sin puertos abiertos.
- **target2 (nginx)**: Contenedor basado en la imagen oficial de Nginx, con el servidor web HTTP escuchando en el puerto 80. Representa un host activo con servicios expuestos.
- **IP inexistente (172.28.0.99)**: Dirección IP no asignada a ningún contenedor dentro de la red. Representa el caso de un host inactivo o inexistente.

2.4. Gestión de dependencias

Las dependencias Python del proyecto se han gestionado con `uv`, una herramienta moderna de gestión de entornos y paquetes para Python que ofrece mayor velocidad

que pip y gestión automática de entornos virtuales. Las dependencias quedan registradas en el fichero `pyproject.toml` ubicado en el directorio `src/`.

Las librerías instaladas para esta práctica son:

- `scapy`: Para la construcción y envío de paquetes de red [2].
- `jupyter`: Para la ejecución del notebook de demostración [3].

3. Descubrimiento de hosts con Scapy

3.1. Introducción a Scapy

Scapy [2] es una librería de Python que permite la construcción, envío, captura y análisis de paquetes de red de forma programática. A diferencia de herramientas como Nmap, Scapy opera a un nivel más bajo, permitiendo al usuario definir con precisión cada campo de cada capa del paquete. Esto la convierte en una herramienta especialmente útil para la investigación, el desarrollo de nuevas técnicas de reconocimiento y la comprensión profunda de los protocolos de red.

Scapy implementa una sintaxis basada en la superposición de capas mediante el operador `/`. Por ejemplo, el paquete `IP(dst="192.168.1.1") / TCP(dport=80, flags="A")` construye un paquete IP con una capa TCP encima, destinado al puerto 80 con el flag ACK activado.

3.2. Protocolos utilizados

3.2.1. UDP (User Datagram Protocol)

UDP es un protocolo de transporte sin conexión. En el contexto del descubrimiento de hosts, se envía un datagrama UDP a un puerto específico del host objetivo. Si el host está activo y el puerto al que se envía está cerrado, el sistema operativo responde con un mensaje ICMP de tipo 3 (Destination Unreachable), código 3 (Port Unreachable). Si el puerto está abierto, la respuesta depende de la aplicación. Si no hay respuesta, el puerto puede estar filtrado o el host puede estar inactivo.

La utilidad de UDP para el descubrimiento de hosts radica precisamente en esta respuesta ICMP: si el host responde con un ICMP Unreachable, sabemos que está activo, aunque el puerto esté cerrado.

3.2.2. TCP ACK

El flag ACK de TCP se utiliza normalmente para confirmar la recepción de datos dentro de una conexión establecida. Sin embargo, cuando se envía un paquete TCP con el flag ACK a un puerto de un host con el que no existe ninguna conexión previa, el sistema operativo responde con un paquete RST (Reset), independientemente de si el puerto está abierto o cerrado. Esta respuesta RST indica que el host está activo.

Esta técnica es especialmente útil porque algunos firewalls permiten el paso de paquetes ACK al considerarlos parte de conexiones establecidas, bloqueando los paquetes SYN, lo que la convierte en una técnica de evasión interesante.

3.2.3. ICMP Timestamp Request

El protocolo ICMP (Internet Control Message Protocol) incluye varios tipos de mensajes. El tipo 13, conocido como Timestamp Request, permite solicitar a un host que devuelva su marca de tiempo actual. Si el host está activo y no filtra este tipo de mensajes, responde con un ICMP Timestamp Reply (tipo 14).

Esta técnica es menos común que el ping ICMP Echo Request (tipo 8) y puede pasar desapercibida en sistemas que solo filtran pings convencionales.

3.3. Implementación de `craft_discovery_pkts`

3.3.1. Diseño de la función

La función `craft_discovery_pkts` ha sido diseñada siguiendo principios de modularidad y reutilización. Acepta los siguientes argumentos:

Argumento	Tipo	Obligatorio	Valor por defecto
`protocols`	str / list	Sí	—
`ip_range`	str	Sí	—
`pkt_count`	dict	No	1 paquete por protocolo
`port`	int	No	80

Tabla 2: Argumentos de la función `craft_discovery_pkts`

La función normaliza el argumento `protocols` a una lista internamente, lo que permite pasarlo tanto como string (un solo protocolo) como lista (varios protocolos). Se limita a un máximo de 3 protocolos simultáneos, tal y como indica el enunciado.

3.3.2. Lógica de construcción de paquetes

Para cada protocolo especificado, la función construye el número de paquetes indicado en `pkt_count` (o 1 si no se especifica). Los paquetes se construyen de la siguiente forma:

- **UDP**: `IP(dst=ip_range) / UDP(dport=port)`
- **TCP ACK**: `IP(dst=ip_range) / TCP(dport=port, flags='A')`
- **ICMP Timestamp**: `IP(dst=ip_range) / ICMP(type=13, code=0)`

La función devuelve una lista con todos los paquetes construidos, que puede ser pasada directamente a las funciones de envío de Scapy.

3.3.3. Función auxiliar `discover_hosts`

Adicionalmente se ha implementado la función `discover_hosts`, que encapsula el flujo completo de descubrimiento: construye los paquetes mediante `craft_discovery_pkts`, los envía usando `sr()` (send and receive) de Scapy, y procesa las respuestas para extraer e imprimir las IPs activas.

La función `sr()` de Scapy devuelve dos listas: los paquetes que han recibido respuesta (`answered`) y los que no la han recibido (`unanswered`). Las IPs activas se extraen del campo `src` de la capa IP de cada respuesta recibida.

3.4. Demostración en Jupyter Notebook

La función ha sido demostrada en un Jupyter Notebook [3] que incluye cuatro escenarios de prueba:

Escenario	IP objetivo	Protocolo	Resultado esperado
Host activo sin servicios	172.28.0.10	TCP ACK	Activo detectado
Host activo con HTTP	172.28.0.11	ICMP Timestamp	Activo detectado
Host inactivo	172.28.0.99	UDP	Sin respuesta
Multi-protocolo	172.28.0.11	TCP ACK + UDP + ICMP	Activo detectado

Tabla 3: Escenarios de prueba del notebook

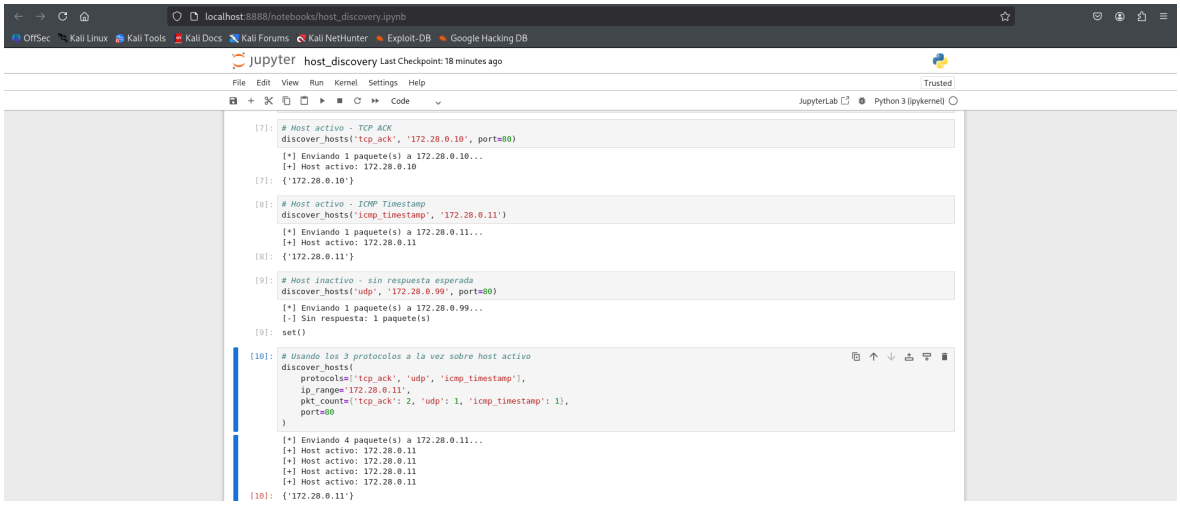

Figura 1: Output del Jupyter Notebook con los resultados del descubrimiento de hosts

3.5. Análisis del tráfico generado

El tráfico generado por Scapy ha sido capturado con tcpdump y analizado con Wireshark [5]. La captura confirma que los paquetes enviados corresponden exactamente con los protocolos especificados y que las respuestas recibidas permiten identificar correctamente los hosts activos.

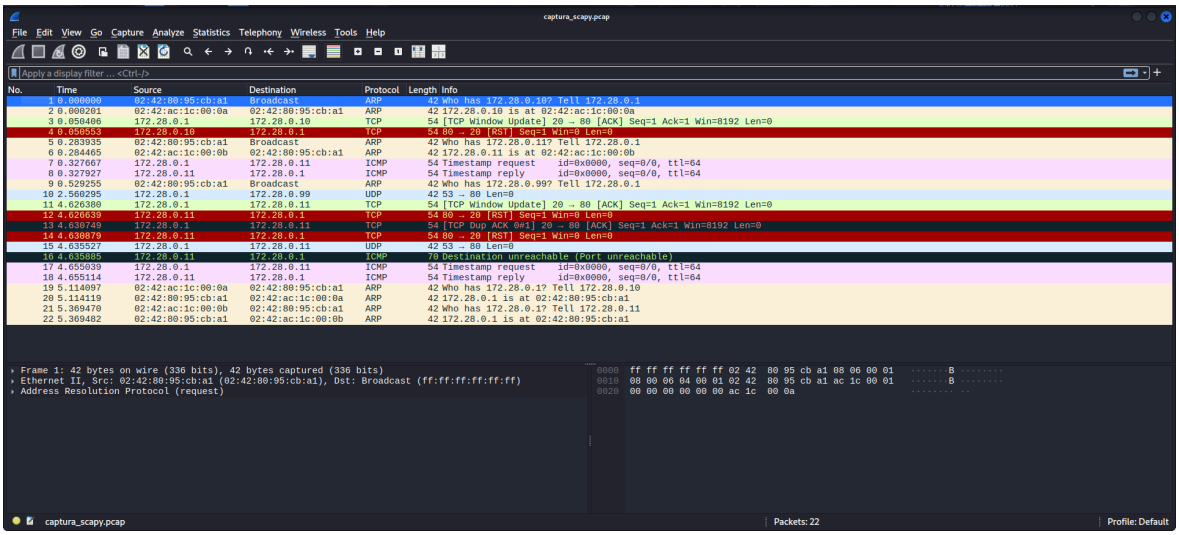

Figura 2: Vista general de la captura de tráfico generada por Scapy en Wireshark

3.5.1. Tráfico TCP ACK

El paquete TCP ACK enviado a `172.28.0.10` tiene el flag ACK activado y está dirigido al puerto 80. El host responde con un paquete RST/ACK, confirmando que está activo.

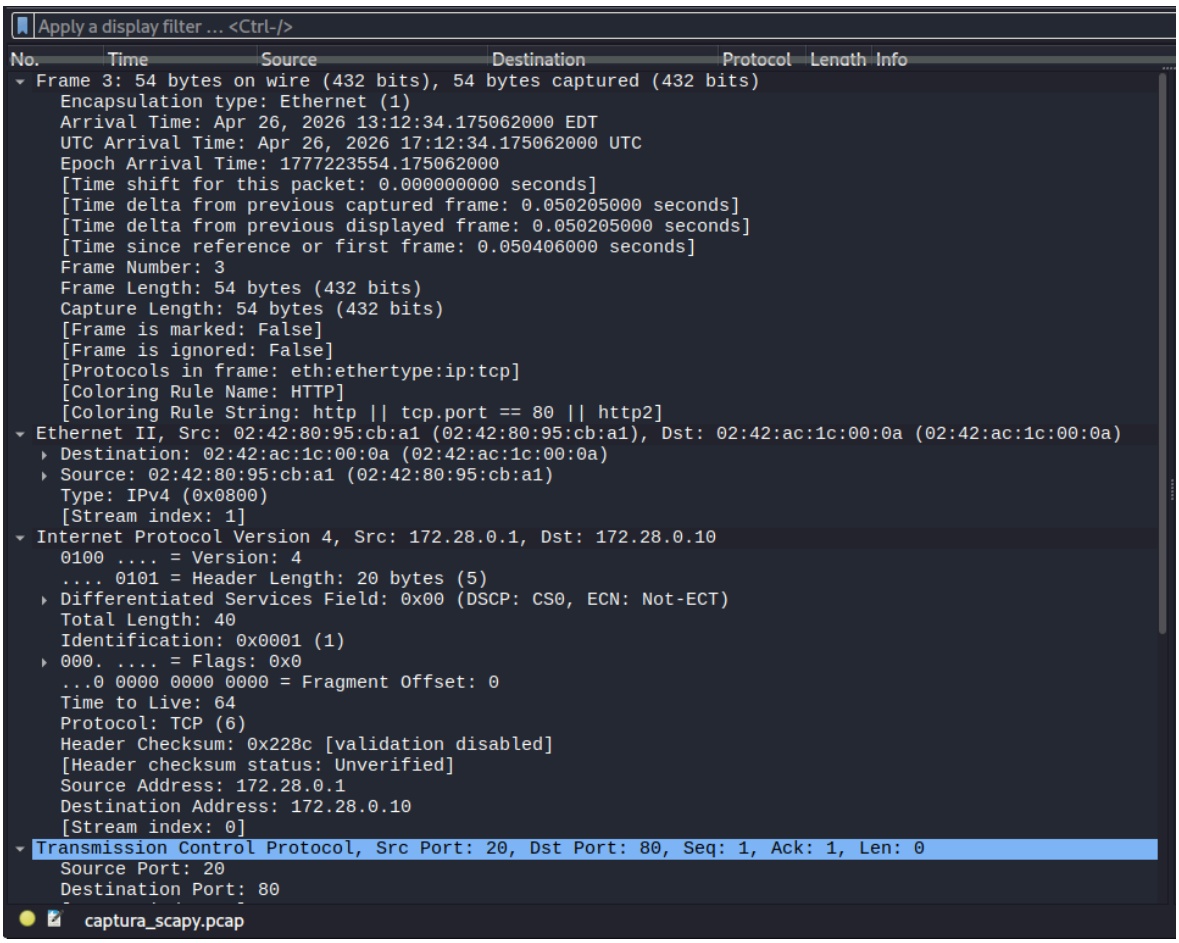

Figura 3: Detalle del paquete TCP ACK enviado a 172.28.0.10

```
▾ Transmission Control Protocol, Src Port: 20, Dst Port: 80, Seq: 1, Ack: 1, Len: 0
    Source Port: 20
    Destination Port: 80
    [Stream index: 0]
    [Stream Packet Number: 1]
  ▸ [Conversation completeness: Incomplete (36)]
    [TCP Segment Len: 0]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 0
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 0
    0101 .... = Header Length: 20 bytes (5)
  ▸ Flags: 0x010 (ACK)
    Window: 8192
    [Calculated window size: 8192]
    [Window size scaling factor: -1 (unknown)]
    Checksum: 0x372d [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ [Timestamps]
  ▸ [SEQ/ACK analysis]
```

Figura 4: Detalle de la respuesta RST/ACK recibida de 172.28.0.10

3.5.2. Tráfico ICMP Timestamp

El paquete ICMP Timestamp enviado a `172.28.0.11` es de tipo 13 (Timestamp Request). El host responde con un ICMP tipo 14 (Timestamp Reply), confirmando que está activo.

```
Apply a display filter ... <Ctrl-/>
No.   Time   Source   Destination   Protocol   Length   Info
▾ Frame 7: 54 bytes on wire (432 bits), 54 bytes captured (432 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Apr 26, 2026 13:12:34.452323000 EDT
    UTC Arrival Time: Apr 26, 2026 17:12:34.452323000 UTC
    Epoch Arrival Time: 1777223554.452323000
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.043202000 seconds]
    [Time delta from previous displayed frame: 0.043202000 seconds]
    [Time since reference or first frame: 0.327667000 seconds]
    Frame Number: 7
    Frame Length: 54 bytes (432 bits)
    Capture Length: 54 bytes (432 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:icmp]
    [Coloring Rule Name: ICMP]
    [Coloring Rule String: icmp || icmpv6]
▾ Ethernet II, Src: 02:42:80:95:cb:a1 (02:42:80:95:cb:a1), Dst: 02:42:ac:1c:00:0b (02:42:ac:1c:00:0b)
  ▸ Destination: 02:42:ac:1c:00:0b (02:42:ac:1c:00:0b)
  ▸ Source: 02:42:80:95:cb:a1 (02:42:80:95:cb:a1)
    Type: IPv4 (0x0800)
    [Stream index: 2]
▾ Internet Protocol Version 4, Src: 172.28.0.1, Dst: 172.28.0.11
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 40
    Identification: 0x0001 (1)
  ▸ 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0x2290 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 172.28.0.1
    Destination Address: 172.28.0.11
    [Stream index: 1]
```

Figura 5: Detalle del paquete ICMP Timestamp Request enviado a 172.28.0.11

```
Internet Control Message Protocol
    Type: 13 (Timestamp request)
    Code: 0
    Checksum: 0x9014 [correct]
    [Checksum Status: Good]
    Identifier (BE): 0 (0x0000)
    Identifier (LE): 0 (0x0000)
    Sequence Number (BE): 0 (0x0000)
    Sequence Number (LE): 0 (0x0000)
    Originate Timestamp: 61589496 (17 hours, 6 minutes, 29.496 seconds after midnight UTC)
    Receive Timestamp: 61589496 (17 hours, 6 minutes, 29.496 seconds after midnight UTC)
    Transmit Timestamp: 61589496 (17 hours, 6 minutes, 29.496 seconds after midnight UTC)
captura_scapy.pcap
```

Figura 6: Detalle del ICMP Timestamp Reply recibido de 172.28.0.11

3.5.3. Tráfico UDP sin respuesta

El paquete UDP enviado a `172.28.0.99` no recibe respuesta, ya que no existe ningún host con esa dirección IP en la red.

```
Frame 10: 42 bytes on wire (336 bits), 42 bytes captured (336 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Apr 26, 2026 13:12:36.684951000 EDT
    UTC Arrival Time: Apr 26, 2026 17:12:36.684951000 UTC
    Epoch Arrival Time: 1777223556.684951000
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 2.031040000 seconds]
    [Time delta from previous displayed frame: 2.031040000 seconds]
    [Time since reference or first frame: 2.560295000 seconds]
    Frame Number: 10
    Frame Length: 42 bytes (336 bits)
    Capture Length: 42 bytes (336 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:udp]
    [Coloring Rule Name: UDP]
    [Coloring Rule String: udp]
Ethernet II, Src: 02:42:80:95:cb:a1 (02:42:80:95:cb:a1), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
  Destination: Broadcast (ff:ff:ff:ff:ff:ff)
  Source: 02:42:80:95:cb:a1 (02:42:80:95:cb:a1)
    Type: IPv4 (0x0800)
    [Stream index: 0]
Internet Protocol Version 4, Src: 172.28.0.1, Dst: 172.28.0.99
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 28
    Identification: 0x0001 (1)
  000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0x2234 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 172.28.0.1
    Destination Address: 172.28.0.99
    [Stream index: 2]
```

Figura 7: Detalle del paquete UDP enviado a 172.28.0.99

```
User Datagram Protocol, Src Port: 53, Dst Port: 80
    Source Port: 53
    Destination Port: 80
    Length: 8
    Checksum: 0xa6bc [unverified]
    [Checksum Status: Unverified]
    [Stream index: 0]
    [Stream Packet Number: 1]
  [Timestamps]
captura_scapy.pcap
```

Figura 8: Ausencia de respuesta al paquete UDP enviado a la IP inexistente

4. Comportamiento por defecto de Nmap y estado de puertos

4.1. Introducción a Nmap

Nmap (Network Mapper) [1] es una herramienta de código abierto ampliamente utilizada en auditorías de seguridad y administración de redes. Permite descubrir hosts, identificar servicios expuestos, detectar sistemas operativos y mucho más. Su versatilidad y potencia la convierten en una herramienta de referencia en el sector.

Sin embargo, su comportamiento por defecto al ejecutarse sin opciones específicas activa una serie de mecanismos preconfigurados que el auditor debe conocer con precisión. En esta sección se analiza en detalle qué hace Nmap por defecto cuando se lanza contra un host.

4.2. Estado de un puerto

El concepto de estado de puerto describe la disponibilidad de un servicio de red en una dirección IP y número de puerto concretos. Nmap [1] define seis estados posibles, aunque los tres principales son:

Estado	Estímulo enviado	Respuesta recibida
Abierto	SYN	SYN/ACK — hay un servicio escuchando
Cerrado	SYN	RST/ACK — no hay servicio, pero el host está activo
Filtrado	SYN	Sin respuesta o ICMP unreachable — firewall bloqueando

Tabla 4: Estados de puerto y mecanismos de detección

4.2.1. Puerto abierto

Un puerto se considera abierto cuando hay una aplicación escuchando activamente en él y aceptando conexiones. Ante un paquete SYN, el host responde con SYN/ACK, indicando su disposición a establecer una conexión TCP. Nmap registra este puerto como abierto e identifica el servicio asociado.

4.2.2. Puerto cerrado

Un puerto cerrado es aquel en el que no hay ninguna aplicación escuchando, pero el host está activo y accesible. Ante un paquete SYN, el sistema operativo responde automáticamente con RST/ACK, rechazando la conexión. Nmap registra este puerto como cerrado.

4.2.3. Puerto filtrado

Un puerto filtrado es aquel en el que un firewall o filtro de paquetes impide que los paquetes lleguen al destino. Ante un paquete SYN, no se recibe respuesta (el paquete

es descartado silenciosamente) o se recibe un mensaje ICMP de tipo unreachable. Nmap registra este puerto como filtrado.

4.3. TCP SYN Scan: el escaneo por defecto de Nmap

Cuando se ejecuta Nmap sin opciones específicas de tipo de escaneo, utiliza el **TCP SYN Scan** (también llamado half-open scan o stealth scan). Este tipo de escaneo funciona de la siguiente manera:

1. Nmap envía un paquete TCP con el flag SYN activado al puerto objetivo.
2. Si el puerto está **abierto**, el host responde con SYN/ACK. Nmap registra el puerto como abierto y envía un RST para cerrar la conexión sin completar el handshake.
3. Si el puerto está **cerrado**, el host responde con RST/ACK. Nmap registra el puerto como cerrado.
4. Si el puerto está **filtrado**, no se recibe respuesta o se recibe un ICMP unreachable.

La ventaja de este tipo de escaneo frente al TCP Connect Scan es que no completa el handshake TCP de tres vías, lo que hace que el escaneo sea más rápido y menos detectable, ya que muchos sistemas no registran conexiones que no llegan a completarse.

4.4. Puertos escaneados por defecto

Por defecto, Nmap escanea los **1000 puertos TCP más comunes**, que corresponden a los puertos con mayor probabilidad de tener servicios activos. Esta lista incluye puertos bien conocidos como el 80 (HTTP), 443 (HTTPS), 22 (SSH), 21 (FTP), 25 (SMTP), entre otros.

En el escaneo realizado contra `172.28.0.11`, Nmap ha identificado:

- **999 puertos cerrados** (respuesta RST/ACK)
- **1 puerto abierto**: el puerto 80/tcp (HTTP), correspondiente al servidor Nginx del contenedor

4.5. Opciones utilizadas en la práctica

Para aislar el tráfico de escaneo de puertos y descartar el tráfico de descubrimiento de hosts y resolución DNS, se han utilizado las siguientes opciones:

- **`-Pn`**: Desactiva el descubrimiento de hosts (ping). Nmap asume que el host está activo y pasa directamente al escaneo de puertos.
- **`-n`**: Desactiva la resolución DNS. Nmap no intenta resolver los nombres de host, evitando tráfico DNS adicional.

4.6. Resultados del escaneo

4.6.1. Escaneo a 172.28.0.11 (nginx — puerto 80 abierto)

```
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ sudo nmap -Pn -n 172.28.0.11
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-26 13:44 EDT
Nmap scan report for 172.28.0.11
Host is up (0.0000070s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
80/tcp open  http
MAC Address: 02:42:AC:1C:00:0B (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.53 seconds
```

Figura 9: Output de Nmap sobre 172.28.0.11 mostrando el puerto 80 abierto

El resultado muestra claramente que el puerto 80/tcp está abierto y el servicio identificado es HTTP. Los 999 puertos restantes aparecen como cerrados.

4.6.2. Escaneo a 172.28.0.10 (alpine — sin puertos abiertos)

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -Pn -n 172.28.0.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-26 13:51 EDT
Nmap scan report for 172.28.0.10
Host is up (0.0000070s latency).
All 1000 scanned ports on 172.28.0.10 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 02:42:AC:1C:00:0A (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.27 seconds
```

Figura 10: Output de Nmap sobre 172.28.0.10 con todos los puertos cerrados

En este caso, los 1000 puertos escaneados aparecen como cerrados, ya que el contenedor Alpine no expone ningún servicio.

4.7. Análisis del tráfico de Nmap con Wireshark

La captura del tráfico generado por Nmap con tcpdump y su posterior análisis en Wireshark [5] confirman el comportamiento descrito. Se observa claramente el patrón característico del TCP SYN Scan: Nmap envía paquetes SYN y recibe RST/ACK de los puertos cerrados.

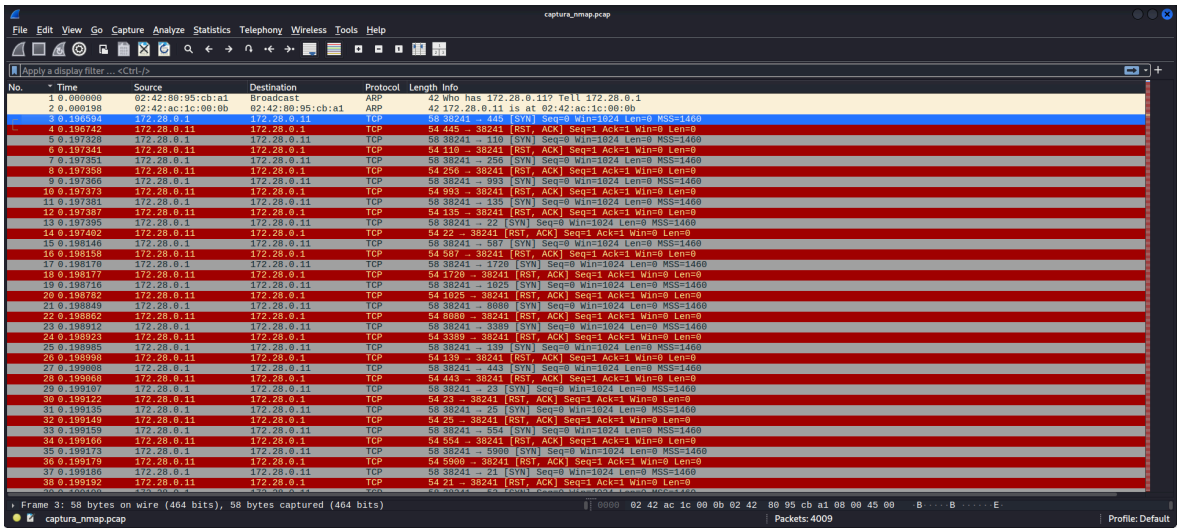

Figura 11: Vista general de la captura de tráfico generada por Nmap — 4009 paquetes capturados

4.7.1. Detalle de un paquete SYN

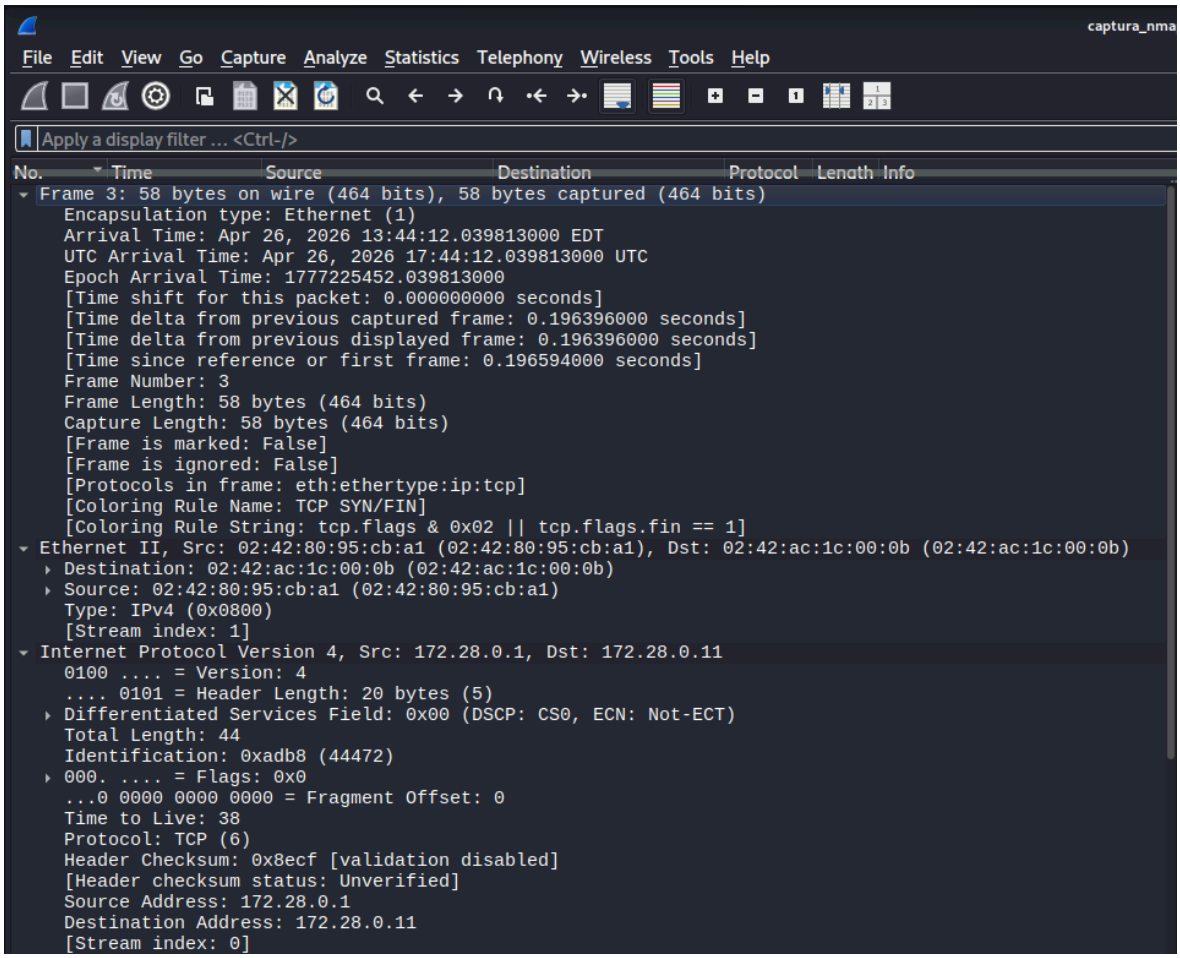

Figura 12: Detalle del paquete SYN enviado por Nmap

```
Transmission Control Protocol, Src Port: 38241, Dst Port: 445, Seq: 0, Len: 0
    Source Port: 38241
    Destination Port: 445
    [Stream index: 0]
    [Stream Packet Number: 1]
  ▸ [Conversation completeness: Incomplete (37)]
    [TCP Segment Len: 0]
    Sequence Number: 0    (relative sequence number)
    Sequence Number (raw): 1577455503
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 0
    Acknowledgment number (raw): 0
    0110 .... = Header Length: 24 bytes (6)
  ▸ Flags: 0x002 (SYN)
    Window: 1024
    [Calculated window size: 1024]
    Checksum: 0x372e [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ Options: (4 bytes), Maximum segment size
  ▸ [Timestamps]
● captura_nmap.pcap
```

Figura 13: Detalle adicional del paquete SYN — flags TCP

4.7.2. Detalle de un paquete RST/ACK (puerto cerrado)

```
No.   Time   Source   Destination   Protocol   Length   Info
Frame 4: 54 bytes on wire (432 bits), 54 bytes captured (432 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Apr 26, 2026 13:44:12.039961000 EDT
    UTC Arrival Time: Apr 26, 2026 17:44:12.039961000 UTC
    Epoch Arrival Time: 1777225452.039961000
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.000148000 seconds]
    [Time delta from previous displayed frame: 0.000148000 seconds]
    [Time since reference or first frame: 0.196742000 seconds]
    Frame Number: 4
    Frame Length: 54 bytes (432 bits)
    Capture Length: 54 bytes (432 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:tcp]
    [Coloring Rule Name: TCP RST]
    [Coloring Rule String: tcp.flags.reset eq 1]
Ethernet II, Src: 02:42:ac:1c:00:0b (02:42:ac:1c:00:0b), Dst: 02:42:80:95:cb:a1 (02:42:80:95:cb:a1)
  ▸ Destination: 02:42:80:95:cb:a1 (02:42:80:95:cb:a1)
  ▸ Source: 02:42:ac:1c:00:0b (02:42:ac:1c:00:0b)
    Type: IPv4 (0x0800)
    [Stream index: 1]
Internet Protocol Version 4, Src: 172.28.0.11, Dst: 172.28.0.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 40
    Identification: 0x0000 (0)
  ▸ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0xe28b [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 172.28.0.11
    Destination Address: 172.28.0.1
    [Stream index: 0]
```

Figura 14: Detalle de un paquete RST/ACK recibido — puerto cerrado

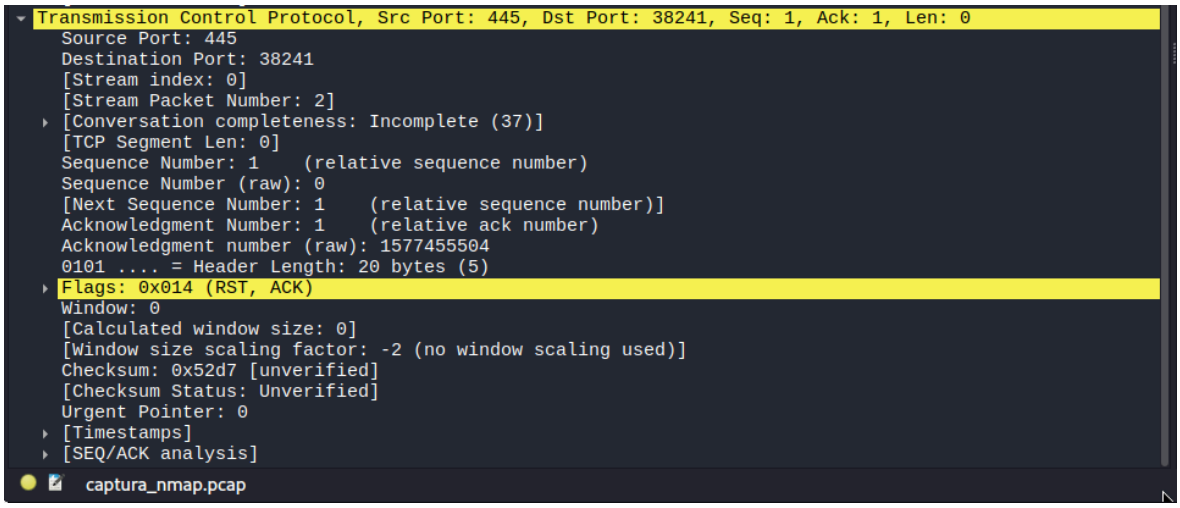

Figura 15: Detalle adicional del paquete RST/ACK

4.7.3. Puerto 80 abierto — filtro Wireshark

Aplicando el filtro `tcp.port == 80` en Wireshark se puede aislar el intercambio de paquetes correspondiente al único puerto abierto detectado:

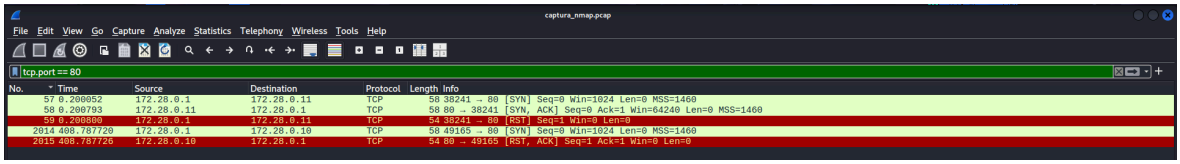

Figura 16: Tráfico filtrado por puerto 80 — intercambio SYN / SYN-ACK / RST

Se observa la secuencia: Nmap envía SYN, el servidor responde SYN/ACK, y Nmap envía RST para cerrar la conexión sin completar el handshake.

5. Resultados

5.1. Resultados del descubrimiento de hosts con Scapy

IP objetivo	Protocolo	Paquetes enviados	Resultado
172.28.0.10	TCP ACK	1	Host activo detectado
172.28.0.11	ICMP Timestamp	1	Host activo detectado
172.28.0.99	UDP	1	Sin respuesta — inactivo
172.28.0.11	TCP ACK + UDP + ICMP	4	Host activo detectado

Tabla 5: Resultados del descubrimiento de hosts con Scapy

La función `craft_discovery_pkts` ha demostrado ser capaz de detectar correctamente hosts activos e inactivos mediante los tres protocolos implementados. El uso simultáneo de múltiples protocolos aumenta la fiabilidad del descubrimiento, ya que un host puede filtrar algunos tipos de paquetes pero no otros.

5.2. Resultados del escaneo de puertos con Nmap

IP objetivo	Puertos escaneados	Puertos abiertos	Puertos cerrados
172.28.0.11	1000	1 (80/tcp HTTP)	999
172.28.0.10	1000	0	1000

Tabla 6: Resultados del escaneo de puertos con Nmap

Estado	Estímulo	Respuesta
Abierto (puerto 80)	SYN	SYN/ACK
Cerrado (resto)	SYN	RST/ACK
Filtrado	SYN	Sin respuesta

Tabla 7: Mecanismos de determinación del estado de puerto por Nmap

El análisis de Wireshark confirma que Nmap ha generado 4009 paquetes en total durante el escaneo de ambos hosts, lo que refleja el volumen de tráfico que genera un escaneo estándar de 1000 puertos por host.

6. Conclusiones

Esta práctica ha permitido profundizar en dos aspectos fundamentales del reconocimiento activo en redes: el descubrimiento de hosts mediante técnicas de bajo nivel con Scapy, y el análisis del comportamiento por defecto de Nmap en el escaneo de puertos.

En cuanto al descubrimiento de hosts, el uso de Scapy [2] como librería de construcción de paquetes a medida ha demostrado ser una alternativa flexible y potente frente a herramientas preconfiguradas. La función `craft_discovery_pkts` implementada permite combinar hasta tres protocolos simultáneamente, configurar el número de paquetes y el puerto de destino, y obtener un conjunto de IPs activas de forma eficiente. La posibilidad de usar múltiples protocolos en paralelo aumenta la robustez del descubrimiento, ya que diferentes hosts pueden responder de forma diferente según su configuración de red y firewall.

En cuanto al análisis de Nmap [1], el estudio del tráfico generado ha confirmado que la herramienta realiza por defecto un TCP SYN Scan sobre los 1000 puertos TCP más comunes. Este comportamiento, aunque altamente eficiente, genera un volumen de tráfico considerable que puede ser detectado por sistemas de detección de intrusiones. El auditor debe ser consciente de este impacto y valorar cuándo utilizar técnicas de escaneo más sigilosas.

El uso de un entorno Docker [4] controlado ha garantizado que todas las pruebas se han realizado de forma ética y legal, sin afectar a sistemas reales. La combinación de tcpdump y Wireshark [5] como herramientas de análisis de tráfico ha permitido verificar y documentar con precisión el comportamiento de cada herramienta utilizada.

Como reflexión final, el reconocimiento activo es una fase que requiere un conocimiento profundo de los protocolos de red y de las herramientas utilizadas. Solo así el auditor puede interpretar correctamente los resultados obtenidos, minimizar el impacto de sus acciones y tomar decisiones informadas durante el proceso de auditoría.

7. Bibliografía

[1] G. F. Lyon, «Nmap Network Scanner». [En línea]. Disponible en: https://nmap.org/

[2] P. Biondi y the Scapy Community, «Scapy: Packet crafting for Python2 and Python3». [En línea]. Disponible en: https://scapy.net/

[3] Python Software Foundation, «Python Programming Language». [En línea]. Disponible en: https://www.python.org/

[4] Docker Inc., «Docker: Accelerated Container Application Development». [En línea]. Disponible en: https://www.docker.com/

[5] Wireshark Foundation, «Wireshark: The World's Most Popular Network Protocol Analyzer». [En línea]. Disponible en: https://www.wireshark.org/